

FRESHMART

SISTEMA ERP — DOCUMENTACIÓN TÉCNICA COMPLETA

1. Descripción General del Proyecto

1.1 Visión y Propósito

FRESHMART ERP es un sistema de gestión empresarial (Enterprise Resource Planning) diseñado específicamente para supermercados y negocios de retail de pequeña a mediana escala. Su objetivo principal es centralizar y digitalizar las operaciones diarias del negocio en una plataforma única, accesible y robusta.

El sistema ha sido diseñado bajo una filosofía de operación offline-first, lo que significa que puede funcionar de manera completamente autónoma sin conexión a internet, garantizando la continuidad operacional en escenarios de falla de conectividad — un requisito crítico para establecimientos comerciales.

1.2 Módulos del Sistema

El sistema está compuesto por cinco módulos integrados que comparten datos en tiempo real:

Dashboard — Panel de control con KPIs, alertas y actividad reciente. Vista personalizada según el rol del usuario.

Punto de Venta (POS) — Gestión de ventas en caja: escaneo de productos, carrito, métodos de pago, emisión de boleta y descuento automático de inventario.

Inventario — Control de stock con alertas de productos críticos, gestión de fechas de vencimiento, márgenes y reposición.

Gestión de Cajeros — Administración de usuarios del sistema: creación, edición, asignación de roles, PIN de acceso y control de estados.

Proveedores — Registro de proveedores, órdenes de compra, historial de transacciones y comparación de precios.

1.3 Requerimientos No Funcionales Clave

2. Arquitectura Técnica

2.1 Stack Tecnológico

2.2 Diagrama de Capas

2.3 Flujo de Autenticación

El sistema implementa un flujo de autenticación en dos factores para operaciones en caja:

El usuario ingresa su usuario y contraseña en la pantalla de login del sistema ERP.

El backend valida las credenciales contra la tabla usuarios en SQLite3 (contraseña hasheada con bcrypt).

Si la autenticación es exitosa, el backend emite un token JWT con el rol y permisos del usuario.

El frontend almacena el token en memoria (no en localStorage para seguridad) y lo incluye en cada petición API.

Para operaciones de apertura/cierre de turno en caja, se solicita adicionalmente el PIN de 4 dígitos del cajero.

El PIN se verifica en el backend — nunca se expone en el frontend.

3. Arquitectura de Base de Datos

3.1 Generalidades

La base de datos utilizada es SQLite3, gestionada a través del módulo nativo sqlite3 de Python. El archivo de base de datos se denomina freshmart.db y se aloja en el directorio de datos del servidor local.

SQLite3 es la elección apropiada para la fase inicial porque no requiere instalación de servidor, permite operación completamente local y offline, tiene excelente rendimiento para volúmenes de hasta ~100.000 registros por tabla, y es nativa en Python sin dependencias adicionales.

3.2 Configuración SQLite3 en Python

3.3 Tablas de la Base de Datos

3.3.1 Tabla: usuarios

Almacena todos los usuarios del sistema (administradores, supervisores y cajeros). Es la tabla central de autenticación y control de acceso.

Tabla: usuarios

3.3.2 Tabla: productos

Catálogo completo de productos del supermercado. Se usa tanto en el POS como en el módulo de inventario.

Tabla: productos

3.3.3 Tabla: ventas

Registro de cada transacción de venta realizada en el POS. Cada venta puede tener múltiples ítems (tabla venta_items).

Tabla: ventas

3.3.4 Tabla: venta_items

Detalle de los productos incluidos en cada venta. Relación de uno a muchos con la tabla ventas.

Tabla: venta_items

3.3.5 Tabla: turnos

Registro de turnos de caja. Cada turno corresponde a un cajero en una sesión de trabajo específica. El arqueo de caja se gestiona al cerrar el turno.

Tabla: turnos

3.3.6 Tabla: inventario_movimientos

Log de todos los movimientos de inventario: ventas, reposiciones, devoluciones y ajustes manuales. Permite trazabilidad completa del stock.

Tabla: inventario_movimientos

3.3.7 Tabla: proveedores

Registro de proveedores con sus datos de contacto, condiciones comerciales y métricas de compra.

Tabla: proveedores

3.3.8 Tabla: ordenes_compra

Registro de órdenes de compra emitidas a proveedores para reposición de inventario.

Tabla: ordenes_compra

3.3.9 Tabla: ordenes_compra_items

Detalle de los productos incluidos en cada orden de compra.

Tabla: ordenes_compra_items

3.3.10 Tabla: alertas_stock

Registro de alertas de stock disparadas. Permite rastrear qué alertas se han enviado por WhatsApp y evitar duplicados.

Tabla: alertas_stock

3.3.11 Tabla: configuracion

Parámetros de configuración del sistema almacenados como clave-valor.

Tabla: configuracion

3.4 Índices Recomendados

Para optimizar las consultas más frecuentes se recomienda crear los siguientes índices:

4. Descripción Funcional de Módulos

4.1 Sistema de Autenticación y Roles

El sistema implementa control de acceso basado en roles (RBAC — Role-Based Access Control). Existen tres roles definidos con permisos diferenciados:

Administrador — Acceso total a todos los módulos. Puede crear, editar y eliminar usuarios, ver todos los reportes y configurar el sistema.

Supervisor — Acceso a POS, Inventario y Proveedores. Puede gestionar stock y órdenes de compra. No puede administrar otros usuarios.

Cajero — Acceso únicamente al Dashboard básico y al Punto de Venta. Su función es procesar ventas.

4.2 Módulo POS — Punto de Venta

El POS es el módulo de mayor frecuencia de uso. Debe ser rápido, intuitivo y funcionar sin interrupciones. El flujo de una venta es el siguiente:

El cajero abre su turno ingresando su PIN de 4 dígitos.

El sistema registra la apertura del turno con el monto de efectivo inicial en caja.

El cajero escanea productos por código de barras o los selecciona desde la grilla de productos por categoría.

El sistema agrega el producto al carrito, verifica stock disponible y aplica descuentos configurados.

Al terminar de agregar productos, el cajero selecciona el método de pago (efectivo, tarjeta, transferencia).

Para efectivo: el cajero ingresa el monto recibido y el sistema calcula el vuelto automáticamente.

Al presionar "Cobrar", el sistema registra la venta, descuenta el stock de cada producto y emite la boleta.

El sistema verifica si algún producto vendido llegó a stock crítico y dispara alerta de WhatsApp si es necesario.

4.3 Módulo de Inventario

Gestiona el catálogo de productos y el control de stock. Las alertas de inventario se clasifican en tres niveles:

Stock Crítico (rojo): Cuando el stock ≤ 5 unidades. Se dispara notificación inmediata por WhatsApp.

Stock Bajo (naranja): Cuando el stock < stock_minimo del producto. Alerta visual en el dashboard.

Próximo a Vencer (morado): Cuando la fecha de vencimiento está a ≤ 7 días. Alerta visual en tabla.

Los movimientos de inventario siempre quedan registrados en la tabla inventario_movimientos, garantizando trazabilidad completa. No es posible modificar el stock directamente — solo a través de ventas, reposiciones o ajustes justificados.

4.4 Módulo de Gestión de Turnos

El manejo de turnos es fundamental para el control de caja y la responsabilidad de cada cajero. El flujo de un turno es:

Apertura de turno: El cajero selecciona su usuario, ingresa su PIN y declara el monto inicial de efectivo en caja (fondo de caja).

Durante el turno: El sistema acumula todas las ventas asociadas al turno activo. El cajero puede ver su resumen parcial en cualquier momento.

Pre-cierre: El supervisor o administrador solicita el cierre del turno. El cajero cuenta el efectivo físico en caja.

Arqueo de caja: El sistema calcula la diferencia entre el efectivo esperado (fondo inicial + ventas en efectivo) y el efectivo contado.

Aprobación: El supervisor revisa el arqueo y aprueba el cierre. Diferencias significativas quedan registradas para seguimiento.

Reporte de turno: Se genera un resumen con totales por método de pago, número de transacciones y resultado del arqueo.

Ejemplo de cálculo de arqueo:

5. Integración WhatsApp Business API

5.1 Propósito y Casos de Uso

El sistema envía notificaciones automáticas por WhatsApp al administrador o supervisor designado cuando ocurren eventos críticos de inventario. Esto permite tomar acciones correctivas incluso cuando el responsable no está físicamente en el local.

Casos de uso implementados:

Stock crítico: Un producto llegó a 5 unidades o menos después de una venta.

Stock agotado: Un producto llegó a 0 unidades.

Producto próximo a vencer: Un producto tiene fecha de vencimiento en los próximos 7 días.

Resumen de turno: Al cierre de cada turno, resumen de ventas y arqueo (opcional, configurable).

5.2 Configuración de la API

Se utiliza la API oficial de WhatsApp Business de Meta (Cloud API). Requiere:

Cuenta de Meta Business Manager verificada.

Número de teléfono registrado en WhatsApp Business Platform.

Access Token permanente generado desde el panel de Meta for Developers.

Plantillas de mensajes (Message Templates) pre-aprobadas por Meta para mensajes de notificación.

5.3 Implementación Python

5.4 Plantillas de Mensaje WhatsApp

Las siguientes plantillas deben ser registradas y aprobadas en el panel de Meta for Developers antes de poder enviar mensajes:

Plantilla: alerta_stock_critico

Plantilla: resumen_turno

6. Endpoints API REST (Backend Python)

6.1 Estructura General

El backend expone una API REST en JSON. Todos los endpoints (excepto /auth/login) requieren el header Authorization: Bearer {token}.

6.2 Endpoints por Módulo

Autenticación

Usuarios / Cajeros

Productos / Inventario

Ventas / POS

Turnos

Proveedores

Dashboard y Reportes

7. Prototipo Actual — Modelo HTML/JS

7.1 Descripción del Prototipo

El prototipo actual es un archivo HTML auto-contenido (index.html) que implementa toda la interfaz de usuario y la lógica de negocio directamente en JavaScript del lado del cliente. No requiere servidor — funciona abriéndolo en cualquier navegador moderno.

7.2 Usuarios de Prueba del Prototipo

7.3 Datos de Ejemplo Incluidos

20 productos en 8 categorías (Lácteos, Panadería, Frutas, Verduras, Carnes, Bebidas, Abarrotes, Limpieza).

5 proveedores con histórico de compras y órdenes de compra (7 órdenes en distintos estados).

Productos con distintos estados de stock: OK, bajo, crítico y próximos a vencer.

Descuentos configurados en algunos productos (5%, 10%, 15%).

Ventas del día pre-cargadas para el dashboard.

7.4 Diferencias entre el Prototipo y el Sistema Final

8. Instalación y Despliegue Local

8.1 Requerimientos de Hardware

El sistema está diseñado para correr en hardware modesto disponible en el local. Las opciones recomendadas son:

Mini PC (NUC, Beelink, etc.): Intel/AMD dual-core 2GHz+, 4GB RAM, 64GB SSD. Opción recomendada para robustez y disponibilidad.

Raspberry Pi 4 (4GB RAM): Opción económica. Requiere SSD externo (no microSD para mayor durabilidad).

PC existente en el local: Si ya existe un equipo encendido permanentemente, puede usarse como servidor.

8.2 Requerimientos de Software

8.3 Configuración del Entorno (.env)

8.4 Inicialización de la Base de Datos

8.5 Arranque del Sistema

8.6 RespalDOS Automáticos

9. Hoja de Ruta de Desarrollo

9.1 Fases del Proyecto

9.2 Consideraciones para Migración a Producción

Migración de SQLite3 a PostgreSQL: El código debe usar SQLAlchemy ORM para facilitar la migración cuando el volumen de datos lo requiera.

Multi-caja: La tabla turnos ya tiene el campo caja_numero para soporte futuro de múltiples cajas simultáneas.

Facturación electrónica SII: Integración con el Servicio de Impuestos Internos de Chile para emisión de boletas electrónicas.

Inventario con lotes: Sistema de lotes (batch numbers) para trazabilidad en productos como carnes y lácteos.

10. Glosario de Términos

FRESHMART ERP — Documentación Técnica v1.0

Generado en Mayo 2026 — Confidencial — Uso Interno